

Edge-Weighted Curl Fields for Analytic Refractive Vector Shaders

A deliberately over-engineered note on turning SVG icons into testable thin lenses

Generated Technical Preprint
compiled for shader pipeline experimentation

April 25, 2026

Abstract

This note formalizes a common “fake refraction” trick used in icon and glyph shaders: sample the background at a nearby coordinate, but make the coordinate offset grow near the silhouette or internal rounded features. The central point is that the visible distortion is not caused by a scalar value being large. It is caused by spatial variation in the offset field. A high but nearly constant field behaves like a translation of the background; a field with a large gradient bends grid lines, shifts features, and creates the impression of glass thickness. We build a compact model from signed distance fields, curvature-weighted boundary bands, a divergence-free curl residual, and a validation texture designed to catch orientation, channel, and support mistakes in WebGPU pipelines. The result is not a physically complete optics simulator, but it is a mathematically inspectable shader contract: every term has a known support, monotonicity expectation, and CPU reference path.

Keywords: signed distance fields, MSDF, WebGPU, WGSL, vector graphics, thin lenses, shader validation, curl noise, glyph rendering.

1 Motivation

A vector icon shader often wants three contradictory things. First, the icon should behave like a crisp SVG mask: the outside is exactly background and the inside is exactly the icon material. Second, the icon should look optically alive: edges appear to refract, rounded corners have thickness, and the interior has a subtle glassy shear. Third, the implementation should be easy to test. A shader that looks good against a photograph can still be flipped vertically, sampling the wrong channel, using a stale atlas page, or confusing pixel and normalized coordinates.

This paper-ish artifact treats the icon as a thin refractive lens defined over a two-dimensional domain $\Omega \subset [0, 1]^2$. The input is not a triangle mesh and not a full volume. It is a scalar shape field derived from an SVG path, typically a signed distance field (SDF) or a multi-channel signed distance field (MSDF). From that scalar we derive a mask, a boundary band, a pseudo-height, and a displacement field. The fragment shader then samples the background

texture at

$$\mathbf{u}' = \mathbf{u} + \Delta(\mathbf{u}), \quad (1)$$

where $\mathbf{u}$ is the screen-space or local icon coordinate and Δ is the refractive offset.

The important distinction is between *field value* and *field gradient*. A large interior value with no gradient can make the icon more opaque, brighter, warmer, or tinted, but it cannot bend a locally constant coordinate system. Distortion comes from non-uniform offsets, and non-uniform offsets usually come from gradients of some height or index field.

2 Notation

Let Ω be the filled vector shape in local UV coordinates. The signed distance field is

$$\phi(\mathbf{u}) = \begin{cases} -d(\mathbf{u}, \partial\Omega), & \mathbf{u} \in \Omega, \\ d(\mathbf{u}, \partial\Omega), & \mathbf{u} \notin \Omega, \end{cases} \quad (2)$$

where $\partial\Omega$ is the boundary. In an ideal SDF, $\|\nabla\phi\| = 1$ almost everywhere away from medial-axis singularities. We use

$$\mathbf{n}(\mathbf{u}) = \frac{\nabla\phi(\mathbf{u})}{\|\nabla\phi(\mathbf{u})\| + \varepsilon} \quad (3)$$

as the outward normal and

$$\mathbf{t}(\mathbf{u}) = (-n_y, n_x) \quad (4)$$

as the tangent. The smooth coverage mask is

$$m(\mathbf{u}) = \sigma\left(-\frac{\phi(\mathbf{u})}{w_m}\right) = \frac{1}{1 + \exp(\phi(\mathbf{u})/w_m)}, \quad (5)$$

where w_m is the antialiasing width in local units.

A boundary band is any non-negative function b concentrated near $\phi = 0$. A useful Gaussian band is

$$b(\mathbf{u}) = \exp\left(-\frac{\phi(\mathbf{u})^2}{2\sigma_b^2}\right). \quad (6)$$

For cartoon glass and icon refraction, the band is typically multiplied by the mask or by a symmetric shell term, depending on whether the outside is allowed to warp.

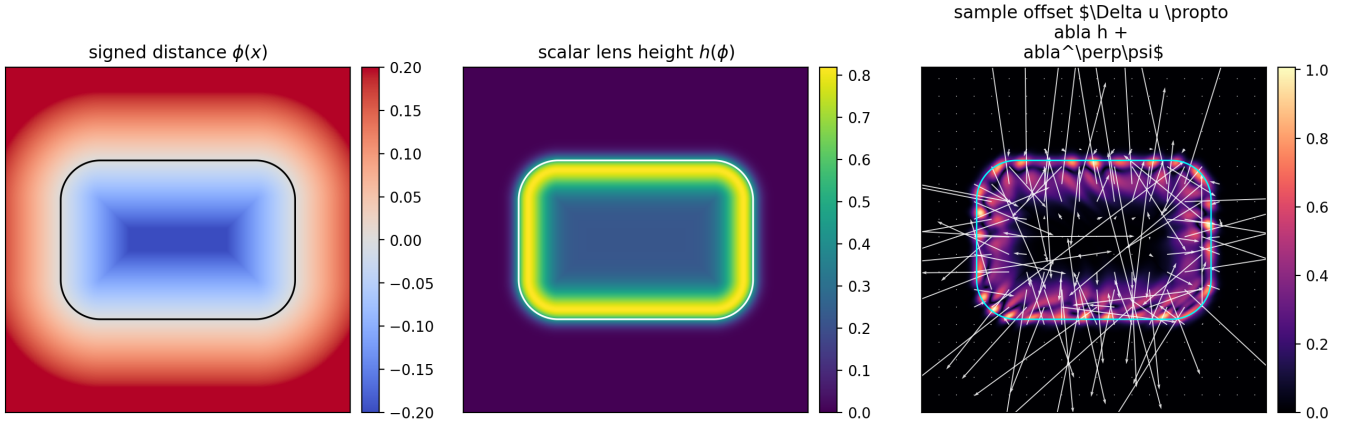


Figure 1: A rounded vector domain produces a signed distance ϕ , a scalar lens height h , and a displacement magnitude. The displacement vectors cluster where h changes. The flat interior can have a high scalar value while producing little visible geometric distortion.

Table 1: Core fields and their intended shader meaning.

Symbol	Meaning
$\mathbf{u}$	local icon coordinate or screen-space sample coordinate
ϕ	signed distance; negative inside the vector shape
m	smooth fill mask used for alpha and support gating
b	boundary band, high near the silhouette
h	scalar pseudo-height or pseudo-index field
Δ	displacement used to sample the background
ψ	scalar stream function whose perpendicular gradient creates swirl
κ	boundary curvature estimate, useful for corner weighting

3 Thin-lens derivation, without pretending it is real optics

A physically inspired derivation is useful because it tells us which derivatives matter. Suppose a thin slab has index of refraction

$$n(\mathbf{u}, z) = 1 + \eta h(\mathbf{u})q(z), \quad (7)$$

where h is a two-dimensional pseudo-height and q integrates to one along the slab thickness. A paraxial ray passing through the slab bends according to the transverse gradient of optical path length:

$$\Delta(\mathbf{u}) \approx C \nabla_{\mathbf{u}} \int n(\mathbf{u}, z) dz = C \eta \nabla h(\mathbf{u}). \quad (8)$$

The constant C absorbs projection, focal distance, texture resolution, and artistic scale. Equation 8 is the justification for the shader pattern “compute a scalar bump, take its gradient, sample the background at an offset.”

This model is incomplete. It ignores Fresnel, multiple interfaces, caustics, total internal reflection, wavelength dependence, and actual surface normals. However, it correctly captures the practical fact that a constant interior

height does not create bending. Only ∇h creates the lens-like coordinate offset.

Proposition 1 (Constant-offset invisibility). *Let $T(\mathbf{u}) = A\mathbf{u} + \mathbf{c}$ be a locally affine validation texture and let $\Delta(\mathbf{u}) = \mathbf{d}$ be constant on a neighborhood U . Then the refracted sample $T(\mathbf{u} + \Delta(\mathbf{u}))$ differs from $T(\mathbf{u})$ by the constant color $A\mathbf{d}$. It contains no local bending, compression, or expansion.*

Proof. Substitution gives $T(\mathbf{u} + \mathbf{d}) = A\mathbf{u} + A\mathbf{d} + \mathbf{c}$. The Jacobian with respect to $\mathbf{u}$ is still A . Thus the local grid orientation and area scale are unchanged. Any visible change is a uniform offset in texture value, not a distortion of the coordinate field. $\square$

The corresponding testable quantity is the Jacobian

$$J(\mathbf{u}) = \frac{\partial(\mathbf{u} + \Delta(\mathbf{u}))}{\partial \mathbf{u}} = I + \nabla \Delta(\mathbf{u}). \quad (9)$$

True local distortion means $\nabla \Delta \neq 0$. Area magnification is roughly $\det J$, and directional stretch comes from the singular values of J .

4 Constructing a useful height field

A robust shader height field should satisfy four constraints.

1. It should be zero or irrelevant outside the shape.
2. It should be smooth enough to avoid shimmering under minification.
3. It should produce large gradients near the silhouette and selected interior ridges.
4. It should not explode at MSDF median discontinuities or at SDF medial axes.

One practical family is

$$h(\mathbf{u}) = m(\mathbf{u}) \left[a_0 + a_1 \exp \left(-\frac{\phi(\mathbf{u})^2}{2\sigma_h^2} \right) \right]. \quad (10)$$

The constant a_0 supplies an interior material value, such as warmth or opacity. The Gaussian term supplies an edge-weighted ridge. Its gradient is

$$\begin{aligned} \nabla h &= \nabla m \left[a_0 + a_1 e^{-\phi^2/(2\sigma_h^2)} \right] \\ &\quad - m a_1 e^{-\phi^2/(2\sigma_h^2)} \frac{\phi}{\sigma_h^2} \nabla \phi. \end{aligned} \quad (11)$$

The first term is strongest at the antialiased boundary; the second is strongest on a ring just inside and outside the boundary depending on support gating. This is the formal version of the intuition that a “roundness gradient” creates the fake refraction: the ring matters because it changes, not because its scalar value is high.

Curvature can exaggerate corners and suppress long flat edges:

$$\kappa(\mathbf{u}) = \nabla \cdot \left(\frac{\nabla \phi}{\|\nabla \phi\| + \varepsilon} \right), \quad (12)$$

with a clipped weight

$$c(\mathbf{u}) = \text{clamp}(1 + \lambda_\kappa \kappa(\mathbf{u}), c_{\min}, c_{\max}). \quad (13)$$

In practice, κ should be prefiltered or estimated at a coarser mip level. Raw curvature from an atlas is noisy and can create crawling highlights.

5 Adding a curl residual

A pure gradient field $\Delta = C\nabla h$ looks like an optical lens but can feel too clinical. A divergence-free residual adds swirl without changing area too aggressively. Let ψ be a scalar stream function, usually a low-frequency procedural noise field. Define

$$\nabla^\perp \psi = \begin{bmatrix} \partial_y \psi \\ -\partial_x \psi \end{bmatrix}. \quad (14)$$

Then

$$\Delta(\mathbf{u}) = s_h \nabla h(\mathbf{u}) + s_\psi b(\mathbf{u}) m(\mathbf{u}) \nabla^\perp \psi(\mathbf{u}). \quad (15)$$

Because $\nabla \cdot \nabla^\perp \psi = 0$ in the continuous limit, the residual is locally incompressible before support gating. The product bm reintroduces divergence near the edge, but it keeps the swirl where users expect optical energy: around silhouettes and ridges.

The stream function should be bandwidth-limited. If the icon occupies N pixels across, a safe maximum frequency is usually below $N/8$ cycles across the domain unless temporal antialiasing or stochastic supersampling is available. For UI icons, the residual should be stationary in local coordinates so the material does not crawl as the camera or page scrolls.

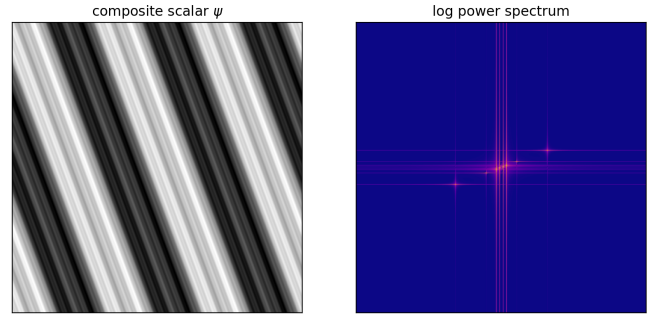


Figure 2: A composite scalar stream function and its log power spectrum. Keeping the residual in low and middle bands reduces shimmer and makes validation possible at finite resolution.

6 The shader as a contract

The WebGPU implementation should expose a small set of resource contracts rather than a bag of magic uniforms. Table 2 describes a minimal version. The most important idea is that every sampled value has a declared coordinate space and every metric has a CPU reference.

Table 2: Resource contract for a testable vector-lens pass.

Resource	Format	Contract
Shape atlas	<i>r8unorm</i> , <i>rg8unorm</i> , or <i>rgba8unorm</i>	Stores SDF, pseudo-SDF, or MSDF. Decode returns signed distance in exactly one declared unit.
Background texture	render attachment or sampled texture	Declares whether $\mathbf{u} = (0, 0)$ is top-left or bottom-left in shader-facing coordinates.
Glyph packet buffer	storage or uniform	Per-instance transform, atlas rectangle, scale, SDF spread, strengths, support mode, and debug flags.
Reference output	storage texture or readback buffer	Optional CPU/GPU comparison target for orientation and numerical tests.
Metrics buffer	storage buffer	Accumulates max error, RMS error, support leakage, and monotonicity violations.

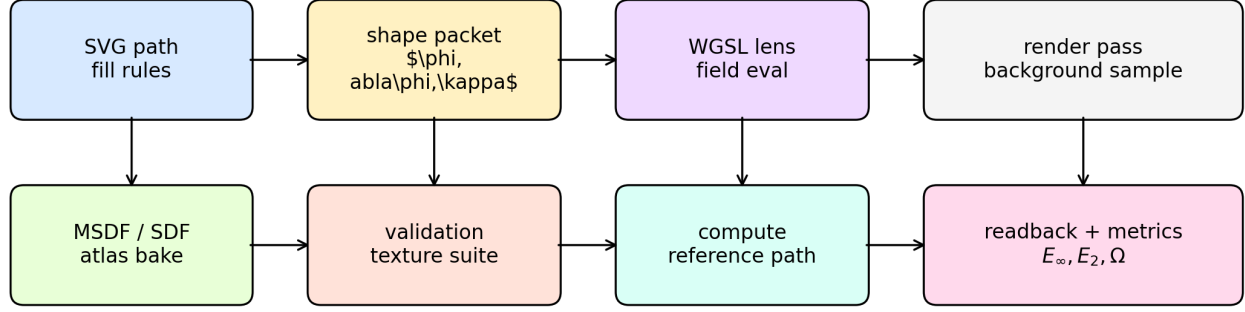
For stable derivatives, compute gradients in the same coordinate system as the displacement. A common bug is to use $\partial\phi/\partial\text{screen}$ in one term and $\partial\phi/\partial\text{local}$ in another. If the instance is non-uniformly scaled, the normal direction and offset magnitude will be wrong. Let M map local coordinates to screen coordinates. Then gradients transform by

$$\nabla_{\text{screen}} h = M^{-T} \nabla_{\text{local}} h. \quad (16)$$

If the shader samples a background in screen UVs, the final offset should be in screen UVs. If it samples an icon-local texture, the offset should remain in local UVs.

7 WGSL evaluation sketch

A WGSL implementation can estimate the scalar gradient by finite differences in atlas UV. Hardware derivatives are useful for antialiasing but should not be the only definition



contract: generated GPU output must match the CPU reference on position, orientation, edge support, and monotone lens response

Figure 3: A complete validation-oriented pipeline. The CPU path does not need to be fast. It needs to be exact enough to reject coordinate flips, wrong support, stale atlas data, and shader algebra regressions.

of the lens normal, because derivative behavior can differ across control flow and helper lanes.

Listing 1: Minimal displacement evaluation. Production code should include mip-aware SDF decode and transform-aware gradients.

```
struct LensParams {
  sdfSpreadPx: f32,
  aaWidth: f32,
  sigmaBand: f32,
  sigmaHeight: f32,
  strengthGrad: f32,
  strengthCurl: f32,
  noiseFreq: f32,
};

fn sigmoid(x: f32) -> f32 {
  return 1.0 / (1.0 + exp(-x));
}

fn decodeSdf(atlasSample: vec4f, p: LensParams) -> f32 {
  // Convention: zero distance is encoded at 0.5.
  // Return signed distance in local UV units.
  return (atlasSample.r - 0.5) * p.sdfSpreadPx;
}

fn samplePhi(uvAtlas: vec2f, p: LensParams) -> f32 {
  let s = textureSample(shapeAtlas, linearSampler, uvAtlas);
  return decodeSdf(s, p);
}

fn heightFromPhi(phi: f32, p: LensParams) -> f32 {
  let m = sigmoid(-phi / p.aaWidth);
  let ridge = exp(-(phi * phi) / (2.0 * p.sigmaHeight * p.
    sigmaHeight));
  return m * (0.22 + 0.78 * ridge);
}

fn streamFunction(localUv: vec2f, p: LensParams) -> f32 {
  let q = localUv * p.noiseFreq;
  return sin(q.x + 0.21 * sin(2.7 * q.y)) *
    sin(q.y + 0.17 * sin(3.1 * q.x));
}

fn displacement(localUv: vec2f, atlasUv: vec2f, texel: vec2f, p:
  LensParams) -> vec2f {
  let phic = samplePhi(atlasUv, p);
  let hxp = heightFromPhi(samplePhi(atlasUv + vec2f(texel.x, 0.0),
    p), p);
  let hxm = heightFromPhi(samplePhi(atlasUv - vec2f(texel.x, 0.0),
    p), p);

```

```
let hyp = heightFromPhi(samplePhi(atlasUv + vec2f(0.0, texel.y),
  p), p);
let hym = heightFromPhi(samplePhi(atlasUv - vec2f(0.0, texel.y),
  p), p);
let gradH = vec2f(hxp - hxm, hyp - hym) / (2.0 * texel);

let band = exp(-(phic * phic) / (2.0 * p.sigmaBand * p.sigmaBand
  ));
let mask = sigmoid(-phic / p.aaWidth);

let psixp = streamFunction(localUv + vec2f(texel.x, 0.0), p);
let psixm = streamFunction(localUv - vec2f(texel.x, 0.0), p);
let psiyp = streamFunction(localUv + vec2f(0.0, texel.y), p);
let psiym = streamFunction(localUv - vec2f(0.0, texel.y), p);
let curlPsi = vec2f(psiyp - psiym, -(psixp - psixm)) / (2.0 *
  texel);

return mask * (p.strengthGrad * gradH +
  p.strengthCurl * band * curlPsi);
}
```

Two details are easy to miss. First, the central difference denominator is vector-valued in the pseudocode above: in real WGSL, divide componentwise by the correct scalar step in each axis. Second, if atlas UV and local UV have different scales, do not use the same texel vector for both. The listing is deliberately compact, not a copy-paste library.

8 Validation texture

The most useful test texture is not pretty. It is diagnostic. It should encode orientation, channel order, and frequency content. A minimal texture stores u in red, v in green, and a checker or band-limited pattern in blue. Then a wrong y-axis convention is immediately visible as a green flip, a red/green channel mix is visible as a color rotation, and a bilinear-vs-nearest bug shows up on the checker.

For an automated test, define the rendered color $C_g(i, j)$

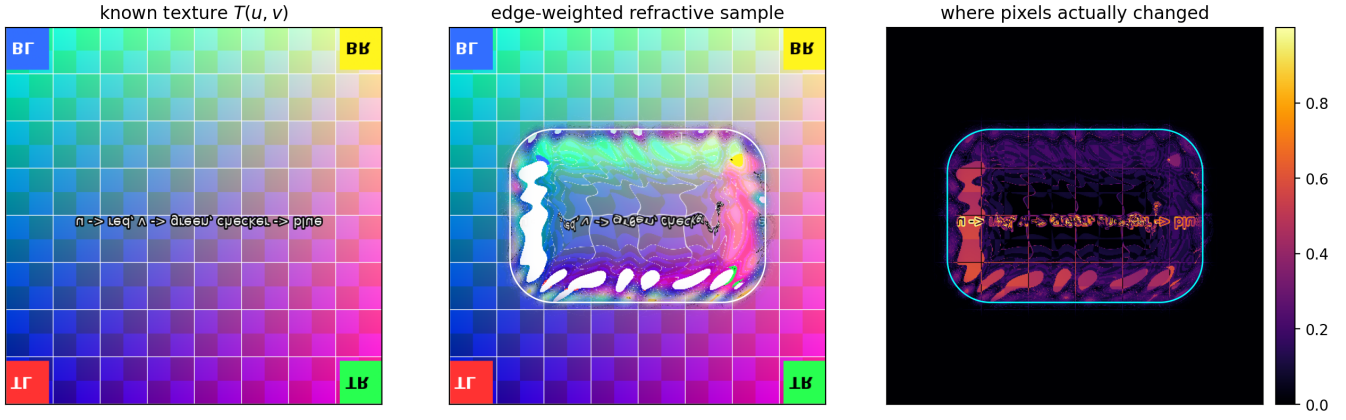


Figure 4: A validation texture and a refracted sample through the rounded vector lens. The right panel shows where pixels actually changed. Notice that change concentrates where the displacement field varies, not merely where the scalar mask is non-zero.

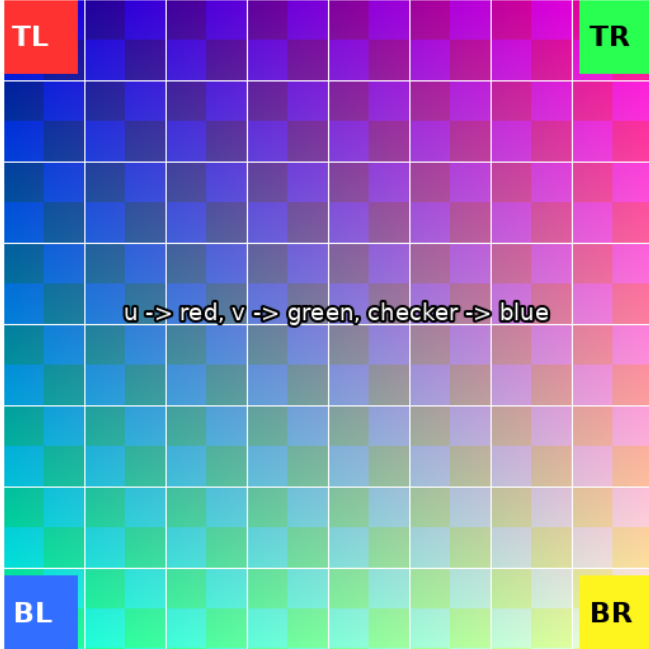


Figure 5: A deliberately obnoxious texture for catching coordinate mistakes. It should be rendered through the shader before any photographic or aesthetic test image.

and CPU reference $C_r(i, j)$ over a finite viewport. Report

$$E_\infty = \max_{i,j} \|C_g(i, j) - C_r(i, j)\|_\infty, \quad (17)$$

$$E_2 = \left(\frac{1}{N} \sum_{i,j} \|C_g(i, j) - C_r(i, j)\|_2^2 \right)^{1/2}, \quad (18)$$

$$\Omega = \sum_{i,j} \mathbb{K}\{m(i, j) < \tau_m \wedge \|C_g - C_{bg}\| > \tau_c\}. \quad (19)$$

Here Ω counts out-of-support leakage: pixels outside the icon that changed when the contract said they should not.

8.1 Metamorphic tests

The pipeline should include tests whose expected output can be inferred without a golden image.

Translation equivariance. If the instance transform is translated by $\mathbf{s}$ and the background is translated by the same amount, the local lens response should be unchanged.

Zero-strength identity. Setting $s_h = s_\psi = 0$ must produce no background warp. Alpha, tint, and material terms may still apply, but the sample coordinate must be exactly the original coordinate within floating-point tolerance.

Constant-height null gradient. Replacing h by a constant must remove the gradient-lens term. This catches accidental use of the scalar height as an offset.

Support monotonicity. Increasing the boundary-band width σ_b should not create displacement arbitrarily far outside the declared support. A test can assert that the changed-pixel set remains within a known dilation of the vector boundary.

Axis flip detection. Rendering the validation texture through an identity lens should preserve the labels TL, TR, BL, and BR. This one test catches a surprising amount of pain.

9 Render-pass architecture

There are two reasonable architectures.

9.1 Direct fragment path

The direct path draws each icon as instanced quads. The fragment shader decodes the atlas, evaluates the displacement, samples the background, applies material terms, and writes the final color. This is simplest and usually fastest for a modest number of icons. It is also the easiest

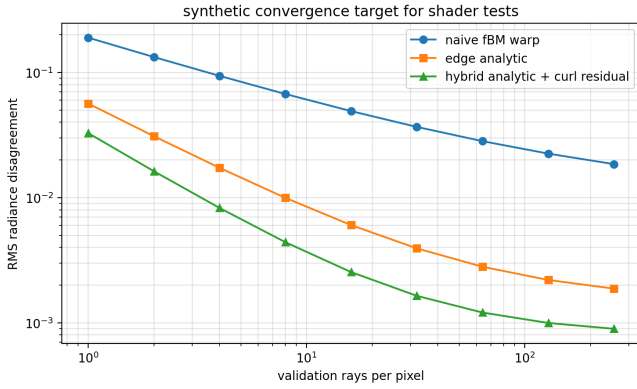


Figure 6: A synthetic example of convergence curves a shader test harness can track. The absolute numbers are not universal; the important part is that each mode has a stable expected trend.

path to keep visually stable because the derivatives are local to the final rasterization.

9.2 Prepass displacement path

The prepass path writes Δ , m , and optional diagnostics into intermediate textures. A later composite pass samples the background and applies the displacement. This costs bandwidth but makes debugging excellent. You can inspect the displacement field as an image, compare it to the CPU reference, and reuse it for multiple materials.

A hybrid path writes only a compact scalar field, such as ϕ or h , and computes final derivatives in the composite pass. This can fail when the scalar was stored at a lower resolution than the final screen footprint. If the atlas is too small, derivatives become quantized and the refraction crawls.

10 Packing vector data

For a single-channel SDF, store the distance in *r8unorm* or *r16float* depending on quality needs. The decode convention should be explicit:

$$\phi = (2S - 1)R_s, \quad (20)$$

where S is the normalized sample and R_s is the signed spread. If the atlas stores distance in pixels, the shader must know the atlas-to-local conversion.

For MSDF, decode the median channel before applying the same lens math:

$$\phi_{msdf} = \text{median}(r, g, b) - 0.5. \quad (21)$$

MSDF is excellent for preserving sharp corners under reconstruction, but it is not automatically a high-quality normal field. The median operation can create derivative discontinuities. One robust compromise is to use MSDF

for coverage and a separate single-channel pseudo-SDF for normals and lens displacement.

For colored SVGs, the refractive lens should be attached to the shape layer, not to final premultiplied color. Otherwise the displacement changes when the artist edits fill color, which makes no geometric sense and breaks validation.

11 Why high value with low gradient looks boring

Assume the background is a smooth image T and the shader samples $T(\mathbf{u} + \Delta(\mathbf{u}))$. A first-order Taylor expansion gives

$$T(\mathbf{u} + \Delta) \approx T(\mathbf{u}) + \nabla T(\mathbf{u})\Delta(\mathbf{u}). \quad (22)$$

This explains why even a constant displacement can be visible against a non-uniform photograph: it samples a nearby color. But the apparent bending of features is controlled by the variation of the coordinate map. Differentiating again,

$$\nabla[T(\mathbf{u} + \Delta(\mathbf{u}))] \approx \nabla T(\mathbf{u})(I + \nabla\Delta(\mathbf{u})). \quad (23)$$

The term $\nabla\Delta$ is what rotates, squeezes, expands, and curves the background. If Δ is derived from h , then $\nabla\Delta$ contains second derivatives of h . This is why a smooth roundness band can look more like glass than a binary mask: it provides a controlled second-derivative profile around the edge.

Lemma 1 (Gradient-only lens). *For the displacement $\Delta = s_h \nabla h$, local magnification is governed by the Hessian of h :*

$$J = I + s_h H_h, \quad (24)$$

where $(H_h)_{ij} = \partial_i \partial_j h$.

The lemma is the shader-level reason that narrow bands feel sharp and wide bands feel soft. A narrow band has larger second derivatives, so it creates stronger local magnification for the same height amplitude.

12 Failure modes

Halo leakage. If displacement is non-zero outside the icon but the material alpha is zero, the background can still warp outside the silhouette. Sometimes that is desired, like heat haze. For an icon, it usually looks like a bug. Gate Δ with m or with an explicit dilated support mask.

Derivative shimmer. Finite differences on an undersampled SDF produce frame-to-frame variation as the icon moves. Mip-aware SDF sampling and stable local coordinates help. For small icons, drop the residual noise and use a prefiltered analytic profile.

Curvature spikes. Corners, cusps, and medial axes can produce extreme curvature estimates. Clamp curvature

weights and, if possible, precompute a curvature proxy from the vector path instead of differentiating the atlas at runtime.

Wrong space. Local gradients, atlas gradients, screen gradients, and background UV gradients are not interchangeable. Write the space into the variable name: `gradH_local`, `offset_screenUv`, `texel_atlasUv`. Boring names prevent expensive bugs.

MSDF median normals. MSDF gives good coverage but can give ugly derivatives. Use median MSDF for the mask and a separate scalar SDF for lens normals if you care about smooth refraction.

13 A small implementation checklist

A production implementation should include the following debug views:

1. raw decoded ϕ in signed colors;
2. mask m ;
3. height h ;
4. ∇h as a vector field;
5. displacement magnitude $\|\Delta\|$;
6. support leakage map;
7. identity render of the orientation texture;
8. CPU difference heatmap.

The shader author should be able to answer these questions from screenshots alone: Is the SDF sign correct? Is y flipped? Does the displacement live inside the intended support? Does zero strength become identity? Are corners over-weighted? Is the residual noise stable under translation? Are the finite differences taken at the expected scale?

14 Conclusion

The visual trick is simple: compute an offset and sample the background somewhere else. The hard part is making the trick principled enough that it can be debugged. A vector-lens shader becomes tractable when the scalar shape field, height profile, gradient field, curl residual, support mask, and coordinate space are separated. The roundness or edge field is not magic by itself. It matters because its derivatives create a non-uniform coordinate map. Once that is explicit, the shader can be tested with ugly diagnostic textures before it is judged against beautiful images.

A Suggested per-instance packet

Listing 2: One possible instance packet layout. Align fields to the backend’s uniform/storage layout rules before shipping.

```
struct VectorLensInstance {
    localToClip0: vec4f;
    localToClip1: vec4f;
    atlasRect: vec4f;      // xy = min, zw = size
    backgroundRect: vec4f; // screen UV rectangle
    sdfSpreadPx: f32;
    aaWidth: f32;
    sigmaBand: f32;
    sigmaHeight: f32;
    strengthGrad: f32;
    strengthCurl: f32;
    curvatureGain: f32;
    supportMode: u32;
    tintPremul: vec4f;
    debugMode: u32;
};
```

B Reference CPU comparison

A CPU reference does not have to reproduce GPU floating-point behavior exactly. It should instead be strict about discrete conventions: atlas rectangle, UV origin, SDF decode, support mask, and sampler mode. A reference renderer can run at low resolution and still catch the majority of broken commits.

Listing 3: CPU-side comparison skeleton.

```
for pixel in viewport:
    local_uv = inverse(instance.local_to_screen) * pixel.center
    atlas_uv = instance.atlas_rect.xy + local_uv * instance.
        atlas_rect.zw
    phi = decode_sdf(bilinear(shape_atlas, atlas_uv))
    h = height_from_phi(phi)
    grad_h = central_difference(height_from_phi, atlas_uv)
    curl = central_difference_stream(local_uv)
    disp = mask(phi) * (k_grad * grad_h + k_curl * band(phi) * curl)
    ref_color = bilinear(background, screen_uv(pixel) + disp)
    gpu_color = readback[pixel]
    accumulate_metrics(ref_color, gpu_color, phi)
```

C Deliberately nerdy extension: wavelength split

For chromatic fake refraction, sample three slightly different offsets:

$$\begin{aligned} C_r &= T_r(\mathbf{u} + \Delta(1 + \delta)), \\ C_g &= T_g(\mathbf{u} + \Delta), \\ C_b &= T_b(\mathbf{u} + \Delta(1 - \delta)). \end{aligned} \tag{25}$$

This is not spectral rendering, but it is a useful art-directable approximation. The same validation texture catches channel order mistakes because the red and green gradients encode known coordinates.

References

- [1] C. Green. Improved alpha-tested magnification for vector textures and special effects. *ACM SIGGRAPH Courses*, 2007.

- [2] K. Perlin. An image synthesizer. *ACM SIGGRAPH Computer Graphics*, 19(3), 1985.
- [3] R. L. Cook. Shade trees. *ACM SIGGRAPH Computer Graphics*, 18(3), 1984.
- [4] E. Bruneton and F. Neyret. Precomputed atmospheric scattering. *Computer Graphics Forum*, 27(4), 2008.
- [5] I. Quilez. Fractal Brownian motion and procedural shader notes. Technical article series.